

SolarBI Pascual: Power BI y Grafana, gobernanza de datos, automatización ETL e IoT

Inteligencia de Negocios - Grupo 50

Nombres:

Mateo Usuga Monroy
Andrés Felipe Ocampo Mesa

Institución Universitaria Pascual Bravo

2026

PARTE A

D1

Grafana es una herramienta para mostrar y vigilar datos. Los data sources son las fuentes de donde toma la información, por ejemplo PostgreSQL. Los panels son los gráficos, tablas o tarjetas que muestran los datos. Los dashboards reúnen varios panels en una pantalla. Las variables permiten elegir valores, como un dispositivo. El alerting genera avisos cuando un dato llega a una condición definida. Grafana OSS se instala y administra en el propio equipo. Grafana Cloud funciona como un servicio en la nube. En SolarBI, Grafana puede servir para que el operador vea la potencia, irradiancia y calidad de los datos de forma rápida.

D2

Criterio	Power BI	Grafana
Propósito principal	Análisis de datos y apoyo a decisiones.	Monitoreo de datos y seguimiento de equipos.
Usuario típico	Analista, encargado de datos o directivo.	Operador, técnico o persona de monitoreo.
Conexión a los datos	Se conecta a muchas fuentes y bases de datos.	Se conecta a diferentes fuentes, como PostgreSQL.
Modelo semántico y lenguaje	Usa un modelo semántico y DAX para medidas y cálculos.	Usa principalmente la consulta propia de la fuente de datos.
Actualización	Puede usar Import, DirectQuery y actualización programada.	Está pensado para consultar y mostrar datos que cambian con frecuencia.
Tiempo real	Puede acercarse al tiempo real según la conexión y configuración.	Es especialmente útil para monitoreo cercano al tiempo real.
Alertas	Puede manejar alertas mediante sus servicios y configuraciones.	Tiene funciones de alertas para avisar cuando se cumplen condiciones.
Licenciamiento y costo	Depende del tipo de licencia y del uso.	Grafana OSS se puede instalar; Grafana Cloud funciona como servicio.
Control de acceso	Permisos, roles y seguridad de datos.	Usuarios, equipos, roles y permisos.

D3

Un dashboard operativo muestra qué está pasando ahora; uno analítico ayuda a comparar y entender los datos; uno estratégico muestra resultados generales para tomar decisiones. En SolarBI, energía, yield y Performance Ratio pueden estar en un dashboard analítico. El ahorro puede ser estratégico porque muestra el beneficio económico. Disponibilidad y alarmas son operativos porque ayudan a reaccionar ante problemas. El porcentaje de datos válidos sirve para controlar la calidad. Para el operador elegiría Grafana por su enfoque de monitoreo. Para un directivo elegiría Power BI por sus análisis y reportes.

D4

En Import, Power BI guarda una copia de los datos y luego la actualiza. En DirectQuery, Power BI consulta la fuente cuando necesita los datos. La actualización programada permite definir cuándo se actualiza el modelo. Para SolarBI usaría Import si se trabaja principalmente con información diaria, porque permite analizar los datos de forma rápida. Si se necesitara consultar información muy reciente, DirectQuery puede ser útil. Grafana está más orientado a mostrar cambios frecuentes y monitorear datos. Por eso usaría Grafana para el seguimiento del operador y Power BI para analizar resultados y ahorro.

D5

Las variables de Grafana y los segmentadores de Power BI sirven para elegir qué información queremos ver. En SolarBI se puede crear una variable llamada dispositivo. Si el usuario selecciona el dispositivo 1, la consulta del panel usa ese valor y muestra solamente sus datos. Si cambia al dispositivo 2, el panel cambia sin crear otro dashboard. En Power BI se puede hacer algo parecido con un segmentador. La diferencia principal es que Grafana puede usar la variable directamente dentro de la consulta que trae los datos, mientras Power BI aplica filtros sobre el modelo.

G1

La gobernanza de datos son las reglas y responsabilidades para decidir cómo se deben manejar los datos. El data management es el trabajo de guardar, organizar, mover, proteger y usar esos datos durante su ciclo de vida. La data

quality revisa si los datos son correctos, completos y confiables. En SolarBI, la gobernanza define reglas para las lecturas; el manejo de datos aplica esas reglas; y la calidad comprueba que la telemetría sea útil. El enfoque DAMA-DMBOK incluye la gobernanza y la calidad dentro de las áreas importantes del manejo de datos.

G2

El data owner es la persona responsable de las decisiones sobre un conjunto de datos. El data steward ayuda a mantener sus reglas, significado y calidad. El data custodian se encarga principalmente de la parte técnica y de proteger los datos. En SolarBI, el data engineer puede apoyar como custodio técnico; el data modeler organiza las estructuras; el BI analyst analiza la información; y el product owner define necesidades del proyecto. Los roles deben quedar claros para saber quién puede decidir, quién revisa la calidad y quién mantiene la parte técnica.

G3

Un data contract es un acuerdo que explica cómo deben llegar los datos y qué se espera de ellos. En SolarBI debe incluir los campos, unidades, frecuencia de llegada, responsable y reglas de calidad. Por ejemplo, debe quedar claro qué significa cada lectura de potencia, irradiancia y temperatura. Si la fuente cambia un campo sin avisar, el ETL puede fallar o interpretar mal la información. Como resultado, el dashboard puede quedar sin datos o mostrar valores incorrectos. Por eso el contrato ayuda a controlar cambios y a mantener estable el flujo de información.

G4

Una single source of truth es una fuente principal y confiable que todos usan para obtener el mismo resultado. Un catálogo de KPIs explica qué significa cada indicador y cómo se calcula. En SolarBI, por ejemplo, el yield puede ser energía producida dividida por potencia instalada y expresarse en kWh/kWp. Otro KPI puede ser datos válidos: filas válidas divididas por filas leídas por 100. Si un KPI se calcula de una manera en DAX y de otra en SQL, Power BI y Grafana pueden mostrar resultados diferentes. Esto puede causar confusión y decisiones equivocadas.

G5

Las lecturas de equipos de SolarBI no son necesariamente datos personales, porque describen equipos y mediciones. Pero si el tablero muestra qué operador atendió cada alarma, aparece información relacionada con una persona. En ese caso se debe controlar quién puede verla y usarla solo cuando sea necesario. El equipo debe manejar esos datos de forma responsable y respetar la Ley 1581 de 2012 cuando exista información personal. En SolarBI, la idea es mostrar solamente la información necesaria para operar la planta y evitar exponer datos personales sin una razón clara.

E1

ETL significa extraer, transformar y cargar los datos. ELT carga primero los datos y después los transforma. En SolarBI, Bronze contiene los datos crudos, Silver los datos limpiados y Gold los datos preparados para el análisis. Power Query puede usarse como herramienta ETL porque permite conectar fuentes, limpiar datos, quitar errores y transformarlos antes de cargarlos al modelo. En este proyecto, revisar rangos, eliminar duplicados y tratar datos faltantes son tareas de transformación y calidad.

E2

Una carga completa vuelve a procesar todos los datos. Una carga incremental procesa solamente los datos nuevos o modificados. Una carga idempotente significa que ejecutar el proceso varias veces no crea duplicados ni cambia el resultado final. En fact_energia_dia, la clave primaria formada por fecha_key y dispositivo_key identifica un día y un dispositivo. Con un UPSERT, si la combinación ya existe se actualiza y, si no existe, se inserta. Así, al ejecutar el ETL dos veces, los conteos deben mantenerse iguales.

E3

Forma de automatización	¿Qué es y qué automatiza?	¿Cuándo conviene?
Cron / Programador de tareas	Ejecuta automáticamente un comando en un horario definido.	Cuando el flujo es sencillo y periódico.
Apache Airflow	Organiza y controla varias tareas de un flujo de datos.	Cuando hay varias tareas y dependencias entre ellas.

Forma de automatización	¿Qué es y qué automatiza?	¿Cuándo conviene?
Actualización programada de Power BI	Actualiza los datos del modelo de Power BI según un horario.	Cuando principalmente se necesita mantener actualizado un informe.

E4

La actualización incremental evita volver a cargar todos los datos. RangeStart y RangeEnd son parámetros que marcan el periodo que Power BI debe procesar. De esta forma, se puede conservar el histórico y actualizar principalmente los datos recientes. Es útil cuando una tabla crece mucho y una actualización completa tarda demasiado. En SolarBI sería conveniente si la telemetría aumenta con el tiempo. Si los datos son pocos, una actualización completa puede ser suficiente y más fácil de manejar.

I1

La arquitectura IoT puede verse en varias capas. Primero está el dispositivo, por ejemplo el inversor, que genera la lectura. Luego está el gateway o edge, que puede recibir y preparar la información. Después está la red o broker, que transporta los datos. Luego está el almacenamiento, como PostgreSQL. Finalmente está la aplicación, como Grafana o Power BI. En SolarBI, la lectura cruda entra en Bronze, después de limpiarse queda en Silver y los datos preparados para análisis quedan en Gold. Finalmente, el dashboard consulta esos datos y los muestra al usuario.

I2

MQTT funciona con un modelo de publicación y suscripción. Un dispositivo publica un mensaje y los clientes suscritos reciben la información. El broker es el intermediario que recibe y distribuye los mensajes. El topic indica dónde se publica la información. QoS 0 busca una entrega rápida sin confirmación; QoS 1 busca que llegue al menos una vez; y QoS 2 busca que llegue exactamente una vez. Para SolarBI se podría usar: pascualbravo/solar///. Así, por ejemplo, se puede separar la información de potencia de cada dispositivo.

I3

Modbus RTU/TCP se usa mucho para comunicar equipos industriales. Modbus RTU normalmente usa comunicación serial y Modbus TCP usa una red TCP/IP. MQTT trabaja con mensajes, topics y un broker, por lo que es común en IoT. OPC UA está pensado para comunicación industrial y permite organizar información de equipos. En una planta solar se puede esperar Modbus en inversores y medidores para obtener lecturas. MQTT puede servir para enviar telemetría hacia otros sistemas. OPC UA puede aparecer en integraciones industriales más completas.

I4

Batch procesa los datos por grupos en momentos definidos. Streaming procesa los datos mientras llegan. En SolarBI, la telemetría llega cada 5 minutos, pero fact_energia_dia tiene información diaria. Al resumir los datos se obtiene una visión más sencilla del total del día y se reduce la cantidad de información. Lo que se pierde es el detalle de cada lectura de 5 minutos. Para las lecturas frecuentes usaría Grafana, porque está orientado al monitoreo y las series de tiempo. Para los resultados diarios usaría Power BI para analizar, comparar y mostrar indicadores.

V1

Git sirve para controlar los cambios de los archivos y GitHub permite guardar repositorios Git y trabajar con otras personas. Un repository es el proyecto; un commit guarda un cambio; una branch es una rama de trabajo; y un pull request permite revisar cambios antes de unirlos. El archivo PBIX es binario y dificulta comparar cambios en Git. PBIP separa partes del proyecto en archivos que se pueden versionar mejor. Para Grafana, el dashboard se puede exportar como JSON y guardar ese archivo en el repositorio. En SolarBI esto ayuda a conservar el historial del trabajo.

REFERENCIAS BIBLIOGRÁFICAS

1. Microsoft. (s. f.). Power BI documentation. Microsoft Learn. <https://learn.microsoft.com/power-bi/>
2. Grafana Labs. (s. f.). Grafana documentation. <https://grafana.com/docs/grafana/>
3. DAMA International. (2017). DAMA-DMBOK: Data Management Body of Knowledge (2nd ed.). Technics Publications.

4. OASIS. (2019). MQTT Version 5.0. OASIS Standard. <https://docs.oasis-open.org/mqtt/mqtt/v5.0/>
5. Congreso de Colombia. (2012). Ley 1581 de 2012: disposiciones generales para la protección de datos personales.

PARTE B — Práctica guiada SolarBI

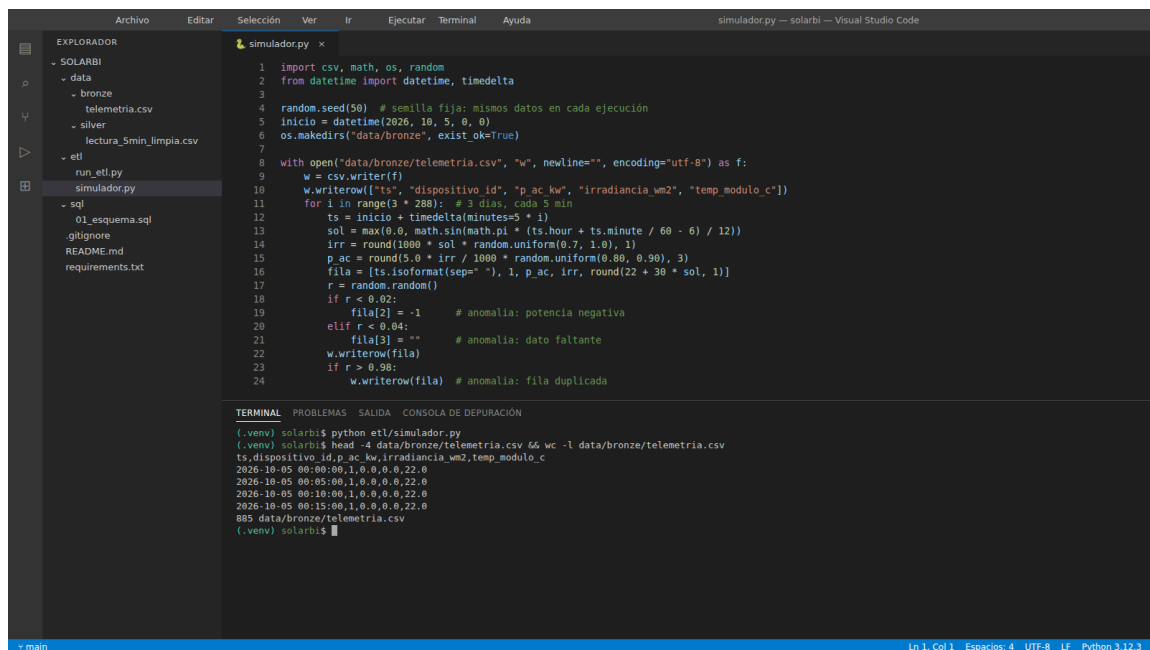
Los mismos datos en dos herramientas: Python, PostgreSQL, Power BI y Grafana

Resumen de lo que hicimos. Simulamos 3 días de telemetría de un inversor de 5 kWp (5 al 7 de octubre de 2026, una lectura cada 5 minutos). Limpiamos los datos con tres reglas de calidad, los cargamos en PostgreSQL y los mostramos en Power BI y en Grafana. Todo se ejecuta con un solo comando y se puede repetir sin duplicar datos.

Indicador	Resultado	Cómo se obtuvo
Filas leídas / válidas	884 / 832	Bronze → Silver
Rechazadas	20 duplicadas · 17 faltantes · 15 fuera de rango	Reglas de calidad
% datos válidos (Silver)	94,12 %	832 / 884
Energía total	80,06 kWh	Suma de potencia × 5/60
Yield	16,01 kWh/kWp	80,06 / 5 kWp
Ahorro estimado	\$ 68.047 COP	80,06 × \$850/kWh

Nota: dos cifras de calidad. El 94,12 % mide filas válidas sobre filas leídas (incluye duplicados en el total). El 96,30 % del tablero es el promedio diario de la tabla fact_energia_dia, que divide entre las 288 lecturas esperadas por día. Se explican ambas para que no parezca una contradicción.

1. Bronze — generar los datos crudos

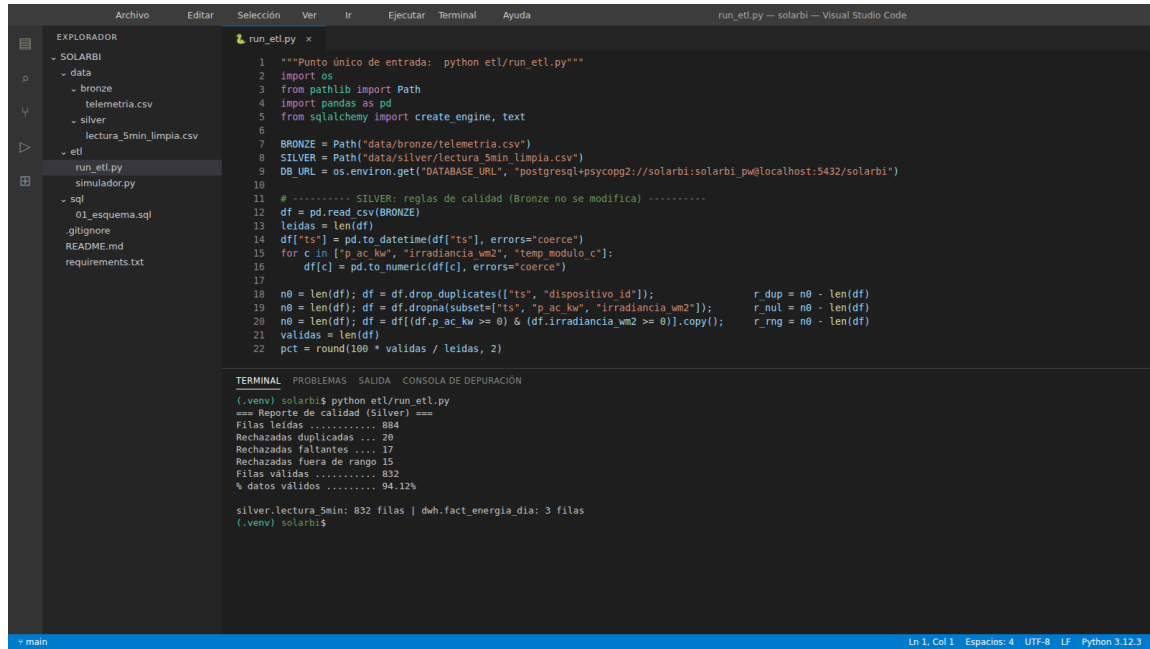


```
1 import csv, math, os, random
2 from datetime import datetime, timedelta
3
4 random.seed(50) # semilla fija: mismos datos en cada ejecución
5 inicio = datetime(2026, 10, 5, 0, 0)
6 os.makedirs("data/bronze", exist_ok=True)
7
8 with open("data/bronze/telemetry.csv", "w", newline="", encoding="utf-8") as f:
9     w = csv.writer(f)
10    w.writerow(["ts", "dispositivo_id", "p_ac_kw", "irradiancia_wm2", "temp_modulo_c"])
11    for i in range(3 * 288): # 3 días, cada 5 min
12        ts = inicio + timedelta(minutes=5 * i)
13        sol = max(0.0, math.sin(math.pi * (ts.hour + ts.minute / 60 - 6) / 12))
14        irr = round(1000 * sol * random.uniform(0.7, 1.0), 1)
15        p_ac = round(5.0 * irr / 1000 * random.uniform(0.80, 0.99), 3)
16        fila = [ts.isoformat(sep=" "), 1, p_ac, irr, round(22 + 30 * sol, 1)]
17        r = random.random()
18        if r < 0.02:
19            fila[2] = -1 # anomalía: potencia negativa
20        elif r < 0.04:
21            fila[3] = "" # anomalía: dato faltante
22        w.writerow(fila)
23        if r > 0.98:
24            w.writerow(fila) # anomalía: fila duplicada
```

```
TERMINAL PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN
(.venv) solarbi$ python etl/simulador.py
(.venv) solarbi$ head -4 data/bronze/telemetry.csv && wc -l data/bronze/telemetry.csv
ts,dispositivo_id,p_ac_kw,irradiancia_wm2,temp_modulo_c
2026-10-05 00:00:00,1,0.0,0.0,22.0
2026-10-05 00:05:00,1,0.0,0.0,22.0
2026-10-05 00:10:00,1,0.0,0.0,22.0
2026-10-05 00:15:00,1,0.0,0.0,22.0
885 data/bronze/telemetry.csv
(.venv) solarbi$
```

Ejecutamos **etl/simulador.py**, que crea **data/bronze/telemetry.csv** con 884 filas. El simulador mete a propósito tres errores: potencia negativa (-1), irradiancia vacía y filas repetidas, para tener qué limpiar. Usamos una semilla fija (*random.seed(50)*) para que cualquiera obtenga los mismos datos. El archivo Bronze no se modifica nunca.

2. Silver — reglas de calidad



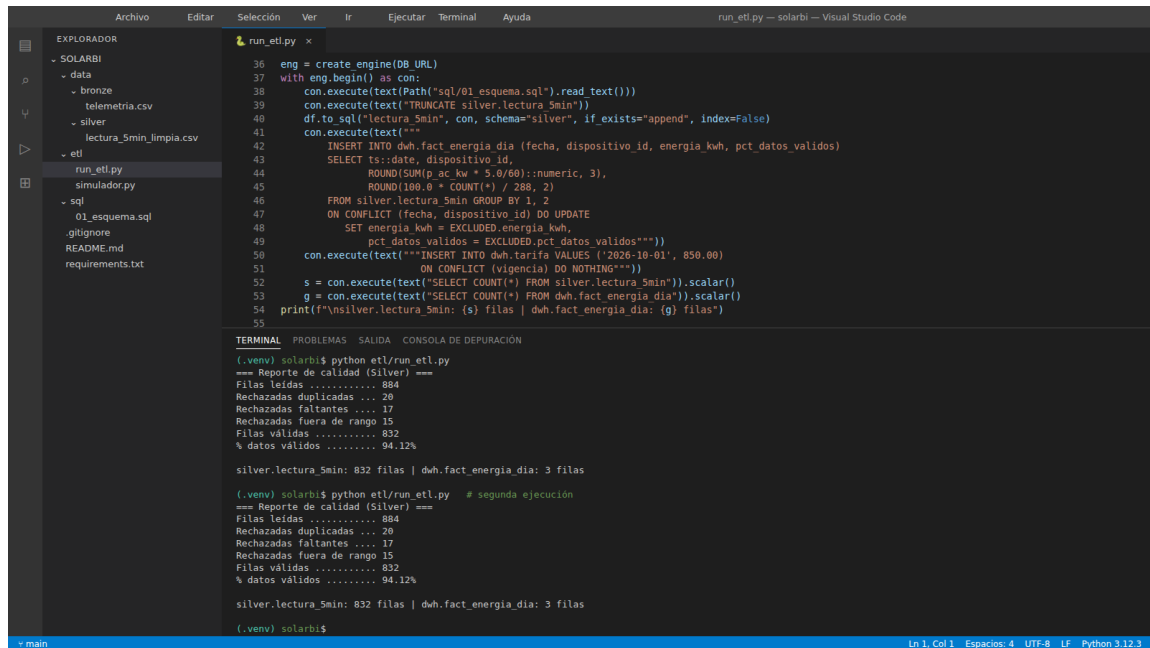
```
1 """Punto único de entrada: python etl/run_etl.py"""
2 import os
3 from pathlib import Path
4 import pandas as pd
5 from sqlalchemy import create_engine, text
6
7 BRONZE = Path("data/bronze/telemetria.csv")
8 SILVER = Path("data/silver/lectura_5min_limpia.csv")
9 DB_URL = os.environ.get("DATABASE_URL", "postgresql-psycopg2://solarbi:solarbi_pw@localhost:5432/solarbi")
10
11 # ----- SILVER: reglas de calidad (Bronze no se modifica) -----
12 df = pd.read_csv(BRONZE)
13 leidas = len(df)
14 df["ts"] = pd.to_datetime(df["ts"], errors="coerce")
15 for c in ["p_ac_kw", "irradiancia_wm2", "temp_modulo_c"]:
16     df[c] = pd.to_numeric(df[c], errors="coerce")
17
18 n0 = len(df); df = df.drop_duplicates(["ts", "dispositivo_id"]); r_dup = n0 - len(df)
19 n0 = len(df); df = df.dropna(subset=["ts", "p_ac_kw", "irradiancia_wm2"]); r_nul = n0 - len(df)
20 n0 = len(df); df = df[(df.p_ac_kw >= 0) & (df.irradiancia_wm2 >= 0)].copy(); r_rng = n0 - len(df)
21 validas = len(df)
22 pct = round(100 * validas / leidas, 2)
```

```
(.venv) solarbi$ python etl/run_etl.py
=== Reporte de calidad (Silver) ===
Filas leídas ..... 884
Rechazadas duplicadas ... 20
Rechazadas faltantes ... 17
Rechazadas fuera de rango 15
Filas válidas ..... 832
% datos válidos ..... 94.12%

silver.lectura_5min: 832 filas | dwh.fact_energia_dia: 3 filas
(.venv) solarbi$
```

En `run_etl.py` aplicamos tres reglas en orden: (1) quitar duplicados por (ts, dispositivo_id), (2) quitar filas con datos faltantes, (3) quitar valores fuera del rango físico (potencia o irradiancia menores a 0). Cada regla cuenta cuántas filas rechaza y al final el programa imprime el reporte de calidad. El resultado limpio se guarda en `data/silver/lectura_5min_limpia.csv`.

3. Gold — carga idempotente en PostgreSQL



```
36 eng = create_engine(DB_URL)
37 with eng.begin() as con:
38     con.execute(text(Path("sql/01_esquema.sql").read_text()))
39     con.execute(text("TRUNCATE silver.lectura_5min"))
40     df.to_sql("lectura_5min", con, schema="silver", if_exists="append", index=False)
41     con.execute(text("""
42         INSERT INTO dwh.fact_energia_dia (fecha, dispositivo_id, energia_kwh, pct_datos_validos)
43         SELECT ts::date, dispositivo_id,
44             ROUND(SUM(p_ac_kw * 3.6/60)::numeric, 3),
45             ROUND(100.0 * COUNT(*) / 288, 2)
46         FROM silver.lectura_5min GROUP BY 1, 2
47         ON CONFLICT (fecha, dispositivo_id) DO UPDATE
48             SET energia_kwh = EXCLUDED.energia_kwh,
49             pct_datos_validos = EXCLUDED.pct_datos_validos"""))
50     con.execute(text("""INSERT INTO dwh.tarifa VALUES ('2026-10-01', 850.00)
51         ON CONFLICT (vigencia) DO NOTHING"""))
52     s = con.execute(text("SELECT COUNT(*) FROM silver.lectura_5min")).scalar()
53     g = con.execute(text("SELECT COUNT(*) FROM dwh.fact_energia_dia")).scalar()
54     print(f"silver.lectura_5min: {s} filas | dwh.fact_energia_dia: {g} filas")
55
```

```
(.venv) solarbi$ python etl/run_etl.py
=== Reporte de calidad (Silver) ===
Filas leídas ..... 884
Rechazadas duplicadas ... 20
Rechazadas faltantes ... 17
Rechazadas fuera de rango 15
Filas válidas ..... 832
% datos válidos ..... 94.12%

silver.lectura_5min: 832 filas | dwh.fact_energia_dia: 3 filas

(.venv) solarbi$ python etl/run_etl.py # segunda ejecución
=== Reporte de calidad (Silver) ===
Filas leídas ..... 884
Rechazadas duplicadas ... 20
Rechazadas faltantes ... 17
Rechazadas fuera de rango 15
Filas válidas ..... 832
% datos válidos ..... 94.12%

silver.lectura_5min: 832 filas | dwh.fact_energia_dia: 3 filas
(.venv) solarbi$
```

Corremos `python etl/run_etl.py` dos veces seguidas y los conteos son iguales: 832 filas en `silver.lectura_5min` y 3 en `dwh.fact_energia_dia`. Eso es una carga idempotente: la tabla de 5 minutos se recarga completa y el resumen diario usa **INSERT ... ON CONFLICT (fecha, dispositivo_id) DO UPDATE**, así que si ya existe el día se actualiza en vez de duplicarse.

3b. Verificación en la base de datos

The screenshot shows the pgAdmin 4 interface. On the left, the tree view shows the database structure: Servers (1) -> PostgreSQL 16 -> Databases (2) -> solarbi -> Schemas (3) -> dwh -> Tables (2) -> fact_energia_dia. The main pane shows a SQL query:

```
SELECT * FROM dwh.fact_energia_dia ORDER BY fecha;  
SELECT (SELECT COUNT(*) FROM silver.lectura_smin) AS silver_filas,  
       (SELECT COUNT(*) FROM dwh.fact_energia_dia) AS fact_filas;
```

Below the query, the 'Data Output' tab shows the results of the query. The first table has 5 columns: fecha date, dispositivo_id integer, energia_kwh numeric(10,3), and pct_datos_validos numeric(5,2). It contains 3 rows of data:

	fecha date	dispositivo_id integer	energia_kwh numeric(10,3)	pct_datos_validos numeric(5,2)
1	2026-10-05	1	27.626	97.22
2	2026-10-06	1	26.393	96.53
3	2026-10-07	1	26.036	95.14

The second table has 2 columns: silver_filas bigint and fact_filas bigint. It contains 1 row of data:

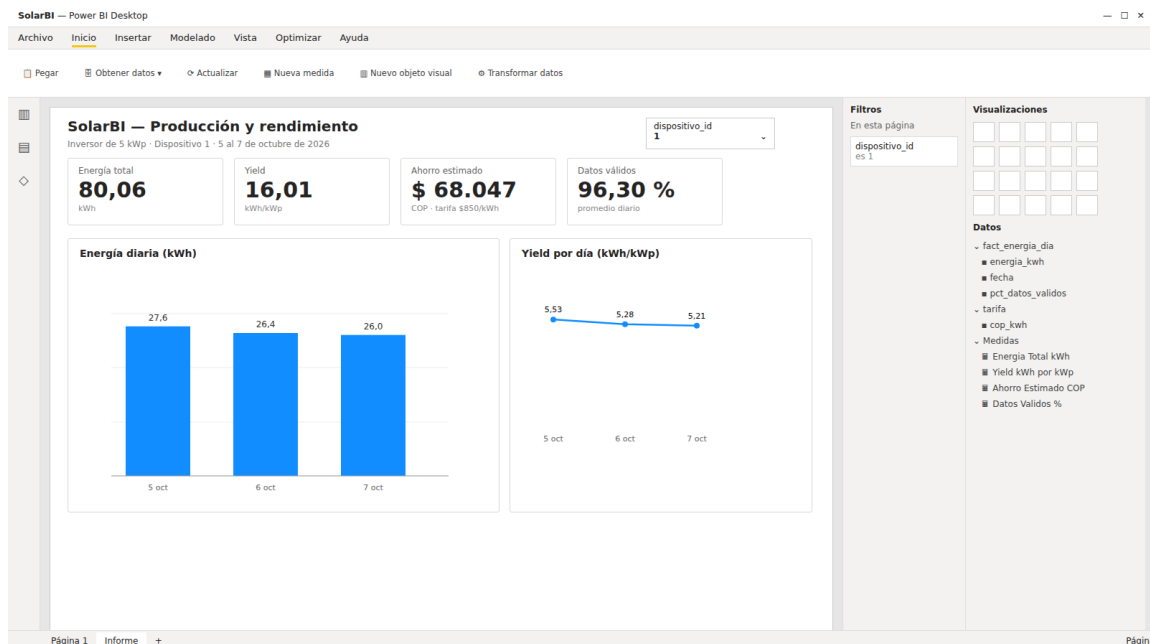
	silver_filas bigint	fact_filas bigint
1	832	3

At the bottom, it says 'Total rows: 1 Query complete 00:00:00.041'.

En pgAdmin comprobamos las tres filas de **dwh.fact_energia_dia** (27,626 · 26,393 · 26,036 kWh) y los conteos finales. La energía de cada intervalo es potencia × 5/60 de hora, y se suma por día. La clave primaria (fecha, dispositivo_id) es la que impide duplicados.

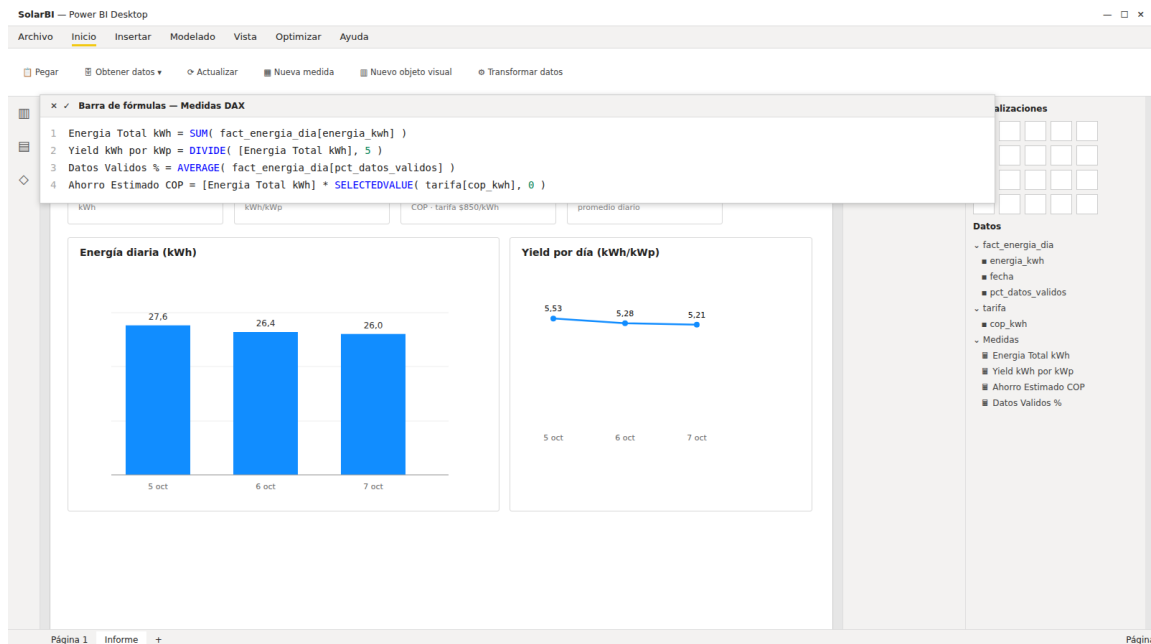
Programación diaria a medianoche (sin implementar). Cron: 0 0 * * * cd /ruta/solarbi && python etl/run_etl.py.
En Windows: Programador de tareas, desencadenador diario a las 00:00, acción: python etl/run_etl.py.

4. Power BI — tablero



Conectamos Power BI Desktop a PostgreSQL (tablas dwh.fact_energia_dia y dwh.tarifa) en modo **Import**, porque los datos son pocos y se actualizan una vez al día. La página tiene la tarjeta de yield, el gráfico de columnas de energía diaria, la tarjeta de ahorro y un filtro por dispositivo. Ninguna cifra está escrita a mano: todas salen de medidas DAX.

4b. Medidas DAX



Las cuatro medidas que alimentan las tarjetas. El ahorro multiplica la energía por la tarifa guardada en la tabla **dwh.tarifa** (850 COP/kWh, valor de referencia que debe reemplazarse por la tarifa real de su operador de energía, citando la fuente en el PDF).

5. Grafana — monitoreo



Agregamos PostgreSQL como data source y creamos una variable **dispositivo** (consulta: `SELECT DISTINCT dispositivo_id FROM silver.lectura_5min`). El panel de series de tiempo usa `$__timeFilter(ts)`, que hace que la consulta respete el rango de tiempo elegido arriba a la derecha, y `${dispositivo}`, que cambia el filtro según la variable. El panel de datos válidos es un gauge con umbrales de color. Al final exportamos el tablero con `Share → Export → JSON` y lo guardamos en `grafana/dashboard.json`.

Consulta del panel de % válidos (misma base, mismo dato que Power BI):

```
SELECT AVG(pct_datos_validos) AS "% válidos"
```

```
FROM dwh.fact_energia_dia
WHERE $__timeFilter(fecha::timestamp) AND dispositivo_id = ${dispositivo};
```

6. Reflexión

Con los mismos datos, Power BI resolvió mejor el análisis de negocio: energía, yield y ahorro salen de medidas DAX claras y se presentan limpias para decidir. Grafana resolvió mejor el monitoreo: muestra las lecturas cada 5 minutos, se actualiza solo y permite ver rápido si el inversor está fallando. Lo que más nos costó fue conectar todo el flujo de punta a punta (simulador, limpieza, base de datos y los dos tableros) y lograr que el proceso se pudiera repetir sin duplicar datos. Para el operador de la planta usaríamos Grafana, porque necesita ver lo que pasa ahora. Para el director usaríamos Power BI, porque necesita resultados resumidos y comparables. Aprendimos además que un mismo KPI debe calcularse en un solo lugar para que ambos tableros muestren la misma cifra.